Practical No :- 01

Aim: Design and implement algorithms to encrypt and decrypt messages using classical substitution and transposition techniques.

1. Caesar Cipher

```
def encrypt(text,shift):
    result = ""

    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    for ch in text:
        if ch in alphabet:
            index = alphabet.index(ch)
            new_index = (index + shift) % 26
            result += alphabet[new_index]

        else:
            result += ch
    return result

def decrypt(text,shift):
    result = ""

    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    for ch in text:
        if ch in alphabet:
            index = alphabet.index(ch)
            new_index = (index - shift) % 26
            result += alphabet[new_index]

        else:
            result += ch
    return result
message = input("Enter message: ").upper()
shift = int(input("Enter shift: "))
cipher = encrypt(message, shift)

cipher=encrypt(message,shift)
print("Encrypted:",cipher)
plain=decrypt(cipher,shift)
print("Decrypted:",cipher)
```

```
Enter message: habiba
Enter shift: 3
Encrypted: KDELED
Decrypted: KDELED
```

2.Rail Cipher

```
def encrypt(text):
    even = ""
    odd = ""

    for i in range(len(text)):

        if i% 2 == 0:
            even += text[i]

        else:
            odd += text[i]
    return even + odd

def decrpyt(cipher):
    mid = (len(cipher) + 1) // 2

    even = cipher[:mid]

    odd = cipher[mid:]

    text = ""

    for i in range (len(odd)):
        text += even[i]
        text += odd[i]

    if len(even)>len(odd):
        text += even[-1]
    return text

msg = input ("Enter Message: ")

enc = encrypt(msg)
print("Encrypted: ",enc)

dec = decrpyt(enc)
print("Decrypted:",dec)
```

```
Enter Message: Habiba
Encrypted:  Hbbaia
Decrypted: Habiba
```

For Key = n

```
def rail_fence_encrypt(text, key):
    if key <= 1:
        return text

    rails = [''] * key

    rail = 0
    direction = 1  # 1 = down, -1 = up

    for char in text:
        rails[rail] += char

        if rail == 0:
            direction = 1
        elif rail == key - 1:
            direction = -1
        rail += direction

    return ''.join(rails)


def rail_fence_decrypt(cipher, key):
    if key <= 1:
        return cipher

    # Mark zigzag positions
    pattern = [['' for _ in cipher] for _ in range(key)]

    rail = 0
    direction = 1

    for col in range(len(cipher)):
        pattern[rail][col] = '*'

        if rail == 0:
            direction = 1
        elif rail == key - 1:
            direction = -1

        rail += direction

    # Fill marked positions
    index = 0
```

```
    for r in range(key):
        for c in range(len(cipher)):
            if pattern[r][c] == '*':
                pattern[r][c] = cipher[index]
                index += 1

    # Read zigzag
    result = []

    rail = 0
    direction = 1

    for col in range(len(cipher)):
        result.append(pattern[rail][col])

        if rail == 0:
            direction = 1
        elif rail == key - 1:
            direction = -1

        rail += direction

    return ''.join(result)

plain_text = input("Enter Plain Text : ")
key = int(input("Enter Key : "))
cipher_text = rail_fence_encrypt(plain_text, key)

print("Plain Text : ", plain_text)
print("Cipher Text : ", cipher_text)
print("Decrypt Text : ", rail_fence_decrypt(cipher_text, key))
```